

Proyecto Final

Curso de Sistemas Operativos y Laboratorio

Título del proyecto

Comparación de rendimiento entre aplicaciones backend concurrentes y secuenciales en lenguajes interpretados y compilados

Miembros del equipo

David Gomez Agudelo y Santiago Trespalacios Bolivar

Resumen

Este proyecto compara el rendimiento de aplicaciones backend desarrolladas en Python (lenguaje interpretado) y en C, Go y Rust (lenguajes compilados), evaluando implementaciones secuenciales y concurrentes. Las métricas analizadas incluyen tiempo de respuesta, throughput, uso de CPU y consumo de memoria. El estudio considera también la influencia del sistema operativo en la gestión de procesos e hilos, con el objetivo de proporcionar evidencia técnica que apoye la toma de decisiones informadas en la selección de tecnologías para sistemas eficientes y escalables.

Introducción

En el desarrollo de aplicaciones backend modernas, el manejo eficiente de múltiples solicitudes concurrentes es un factor crítico para garantizar rendimiento, disponibilidad y escalabilidad. La diversidad de lenguajes de programación y paradigmas de ejecución genera incertidumbre en la selección de tecnologías, especialmente cuando el rendimiento es un requisito fundamental; con frecuencia, estas decisiones se basan en preferencias o tendencias del mercado en lugar de evaluaciones objetivas [5].

En el contexto actual de microservicios y sistemas distribuidos, elegir el lenguaje y modelo de ejecución adecuados impacta directamente en la escalabilidad, la eficiencia en el uso de recursos (CPU y memoria) y los costos de infraestructura. Estudios como TechEmpower Web Framework Benchmarks [3] y trabajos presentados en NOBUGS 2024 [4] han evidenciado diferencias significativas entre lenguajes compilados e interpretados bajo carga concurrente, justificando un análisis experimental controlado como el propuesto. El

framework de benchmarking Wasure [11] también destaca la importancia de metodologías controladas y herramientas especializadas para evaluar el rendimiento en distintos entornos de ejecución.

Antecedentes o marco teórico

1. Lenguajes Interpretados vs Compilados

Los lenguajes compilados (C, Go, Rust) transforman el código fuente en código máquina antes de la ejecución, permitiendo instrucciones directas al procesador con menor overhead. Los lenguajes interpretados, como Python, traducen el código en tiempo de ejecución instrucción a instrucción, añadiendo ciclos adicionales de procesamiento. Como señalan Desai [9] y Xenoss [10], esta diferencia estructural produce brechas de rendimiento medibles, especialmente en tareas de cómputo intensivo. Rust aplica verificaciones de ownership y borrowing en tiempo de compilación [9], mientras que Go destaca por su compilación rápida con tan solo 25 palabras clave [10].

2. Concurrencia y Paralelismo (Diferenciación clave)

La concurrencia se refiere a la capacidad de gestionar múltiples tareas en progreso simultáneo, mientras que el paralelismo implica la ejecución física simultánea en múltiples núcleos.

3. Planificación del CPU y Gestión de Procesos (Relación con Sistemas Operativos)

La interacción entre las aplicaciones backend y el sistema operativo es determinante para el rendimiento. El scheduler del sistema operativo decide qué proceso o hilo recibe tiempo de CPU y cuándo, lo que afecta directamente la latencia y el throughput de los servicios. En Linux, el Completely Fair Scheduler (CFS) distribuye el tiempo de CPU entre los procesos activos de forma proporcional a su prioridad, lo que influye en el comportamiento de aplicaciones concurrentes con múltiples hilos.

4. Manejo de Memoria

C ofrece control manual mediante malloc/free, logrando máxima eficiencia pero requiriendo responsabilidad del programador. Rust elimina fugas y condiciones de carrera mediante su modelo de ownership, sin necesitar recolector de basura [9]. Go utiliza un recolector mark-and-sweep concurrente que puede introducir pausas ocasionales [10]. Python combina conteo de referencias con recolección de basura, estrechamente ligado al GIL [8].

5. Antecedentes empíricos

Los resultados de TechEmpower Web Framework Benchmarks (ronda 23, febrero 2025) [3] muestran que los frameworks en Rust y C superan consistentemente a los interpretados en

throughput. Netguru (2025) [6] reporta que servidores en Rust procesaron aproximadamente 60.000 solicitudes/segundo frente a 40.000 de Go, con latencias promedio de 15 ms y 20 ms respectivamente bajo 1.000 solicitudes concurrentes. El estudio NOBUGS 2024 [4] confirma la ventaja de Rust y Go en tareas CPU-intensivas, mientras que Python puede ser competitivo en escenarios I/O-bound de baja concurrencia.

Objetivos

Objetivo General:

Comparar el rendimiento de aplicaciones backend implementadas de forma secuencial y concurrente en lenguajes interpretados y compilados, mediante la medición y análisis de métricas de desempeño bajo condiciones controladas de ejecución.

Objetivos Específicos:

- Implementar un mismo servicio backend funcional en los lenguajes de programación Python, C, Go y Rust, manteniendo equivalencia en la lógica y funcionalidad del sistema.
- Desarrollar versiones secuenciales y concurrentes de cada implementación, utilizando los mecanismos de concurrencia propios de cada lenguaje.
- Diseñar y ejecutar pruebas de carga controladas que permitan simular múltiples solicitudes concurrentes hacia los servicios backend.
- Medir y registrar métricas de rendimiento relevantes, tales como: tiempo de respuesta, throughput (solicitudes por segundo), uso de CPU o consumo de memoria.
- Analizar las diferencias en el consumo de recursos y el comportamiento del sistema bajo diferentes niveles de concurrencia.
- Evaluar la escalabilidad de cada implementación en función del incremento de la carga de trabajo y del número de solicitudes concurrentes.

Metodología

El proyecto sigue un enfoque experimental comparativo. Se desarrollarán versiones funcionalmente equivalentes del mismo servicio backend en los cuatro lenguajes, cada uno con una implementación secuencial y una concurrente. El servicio recibirá solicitudes por red, procesa una operación definida y generará una respuesta, simulando el comportamiento típico de un backend real.

Las pruebas se ejecutarán en un sistema Linux (Ubuntu 24.04 LTS) con hardware documentado (número de núcleos, RAM y arquitectura). Los procesos de prueba serán aislados para minimizar interferencias de otros servicios del sistema. Para la generación de carga se utilizará wrk como herramienta principal, complementado con Apache Benchmark (ab) para validación. Para el monitoreo de métricas se emplearán: perf para uso de CPU e instrucciones por ciclo, /proc/meminfo y vmstat para consumo de memoria, y las propias herramientas de carga para tiempo de respuesta y throughput.

Por otro lado, las pruebas combinarán los niveles de concurrencia, totales de solicitudes

y duraciones descritos en la Tabla 1. Cada escenario se repetirá entre 3 y 5 veces bajo las mismas condiciones, reportando el promedio y la desviación estándar para garantizar validez estadística.

Diseño de Experimentos

1. Hipótesis

Las aplicaciones backend implementadas en lenguajes compilados tenderán a presentar un mejor rendimiento que aquellas desarrolladas en lenguajes interpretados, especialmente en escenarios de alta concurrencia, debido a una ejecución más eficiente y una menor sobrecarga en la gestión de recursos del sistema. Dentro de los lenguajes compilados, aquellos con gestión de memoria más eficiente y tipos de datos más cercanos al hardware, como Go o Rust, suelen superar en rendimiento a otros lenguajes compilados.

2. Variables del experimento

- Independientes: lenguaje de programación (Python, C, Go, Rust), modelo de ejecución (secuencial / concurrente) y nivel de concurrencia (1, 10, 50, 100 solicitudes simultáneas).
- Dependientes: tiempo de respuesta (ms), throughput (solicitudes/segundo), uso de CPU (%), consumo de memoria (MB y %) y tiempo total de ejecución (s).
- Controladas: sistema operativo, hardware, configuración de red, tipo de servicio backend, tamaño de solicitudes y herramientas de prueba.

3. Validación

La validación funcional consistirá en verificar que todas las implementaciones produzcan respuestas correctas y equivalentes ante las mismas solicitudes, y que las versiones concurrentes no generen errores de sincronización ni pérdida de datos. La validación de rendimiento se realizará mediante pruebas de carga que registren las métricas definidas, permitiendo una comparación objetiva entre implementaciones.

4. Escenarios experimentales

El experimento se realizará combinando los siguientes valores de las variables independientes, generando escenarios que representan condiciones reales de operación con carga progresiva:

Nivel de concurrencia	Total de solicitudes	Duración de prueba
1, 10, 50, 100	100 / 1,000 / 10,000	30 s / 60 s

Tabla 1. Escenarios experimentales

5. Repetición de experimentos

Para reducir la variabilidad y mejorar la confiabilidad estadística de los resultados, cada escenario experimental se ejecutará entre 3 y 5 veces bajo las mismas condiciones, reportando el valor final como el promedio de todas las repeticiones.

6. Análisis de resultados

Los datos recopilados se analizarán mediante comparaciones tabulares y gráficas entre implementaciones, identificando diferencias estadísticamente significativas en las métricas de rendimiento.

Cronograma

•Link [Cronograma](#)

Referencias

- [1] Beazley, D. (2009). Inside the Python GIL. PyCon 2009.
<http://www.dabeaz.com/python/GIL.pdf>
- [2] Shannon, M. (PEP 703). Making the Global Interpreter Lock Optional in CPython. Python Software Foundation. <https://peps.python.org/pep-0703/>
- [3] TechEmpower. (2025). Web Framework Benchmarks – Round 23.
<https://www.techempower.com/benchmarks/>
- [4] NOBUGS 2024 Conference. Performance Benchmarking of Node.js, Rust, Go, and Python in Web Applications. ESRF. <https://indico.esrf.fr/event/114/contributions/777/>
- [5] Çelik, F. (2026). Golang vs Rust vs Python – Battle of Backend. DEV Community.
<https://dev.to/firircelik/golang-vs-rust-vs-python-battle-of-backend-can>
- [6] Netguru. (2025). Golang vs Rust: Which Language Wins for Backend in 2025?
<https://www.netguru.com/blog/golang-vs-rust>
- [7] Pham, T. A. (2025). Best Popular Backend Frameworks by Performance. DEV Community.
<https://dev.to/tuananhpham/popular-backend-frameworks-performance-benchmark-1bkh>
- [8] Real Python. (2024). What Is the Python Global Interpreter Lock (GIL)?
<https://realpython.com/python-gil/>
- [9] Desai, J. (2026). Performance Comparison of Python, Golang, Rust, and C++.
<https://jinaldesai.com/performance-comparison-of-python-golang-rust-and-c/>
- [10] Xenoss. (2025). Rust vs Go vs Python: The Performance Language Showdown.
<https://xenoss.io/blog/rust-vs-go-vs-python-comparison>
- [11] J. Doe et al., "Wasure: A Modular Toolkit for Comprehensive WebAssembly Benchmarking," arXiv preprint arXiv:2602.05488, 2026. Available: <https://arxiv.org/abs/2602.05488>